HDOC Day-61 Activity

Two Sum Using Pointers and Nested Loops

Student Name	Dhruv Pandey
SAP ID	590037660
Batch	15
Course / Activity	HDOC Day-61

Aim

To write a C program that finds two distinct indices in an integer array whose corresponding values add up to a given target. The array is passed to **two_sum()** using its address and accessed through pointers.

Problem Requirements

- Read the size and positive integer elements of a variable-length array **nums[]**.
- Read an integer **target**.
- Pass the array to **two_sum()** using its address.
- Find distinct indices i and j such that **nums[i] + nums[j] = target**.
- Display the two indices separated by a space.
- Display **-1 -1** if no valid pair exists.
- Use a nested-loop approach.
- Assume at most one valid solution exists.

Algorithm

1. Read n, the n elements of nums[], and target.
2. Pass nums, n, and target to **two_sum(nums, n, target)**.
3. Use an outer loop i from 0 to n-2.
4. For each i, use an inner loop j from i+1 to n-1. Starting at i+1 guarantees that i and j are distinct.
5. Check whether ***(nums+i) + *(nums+j) == target**.
6. If true, print i and j and return immediately.
7. If all pairs are checked without finding a solution, print -1 -1.

C Program

```c
#include <stdio.h>

void two_sum(int *nums, int n, int target)
{
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            if (*(nums + i) + *(nums + j) == target)
            {
                printf("%d %d", i, j);
                return;
            }
        }
    }

    printf("-1 -1");
}

int main()
{
    int n, target;

    printf("Enter the size of array: ");
    scanf("%d", &n);

    int nums[n];

    printf("Enter the elements: ");
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &nums[i]);
    }

    printf("Enter target: ");
    scanf("%d", &target);

    two_sum(nums, n, target);

    return 0;
}
```

Pointer Usage

two_sum(nums, n, target) passes the address of the first element. The function parameter **int *nums** receives that address. ***(nums+i)** is equivalent to nums[i], and ***(nums+j)** is equivalent to nums[j].

Example 1 Dry Run

i	j	Values	Sum	Target 9?
0	1	2 + 7	9	Yes

At indices 0 and 1, the values are 2 and 7. Their sum is 9, so the function immediately prints **0 1**.

Test Cases / Expected Results

Case	Input Array	Target	Expected Output
Example 1	2 7 11 15	9	0 1

Example 2	3 2 4	6	1 2
Example 3	3 3	6	0 1
Example 4	1 2 3	7	-1 -1

Example 2 Explanation

For **3 2 4** with target 6, the pair at indices 1 and 2 contains values 2 and 4. Since 2 + 4 = 6, the output is **1 2**.

Example 3 - Equal Values at Different Indices

The array is **3 3**. Although both values are equal, they are stored at different indices 0 and 1. Because 3 + 3 = 6, the valid output is **0 1**.

Example 4 - No Solution

For **1 2 3** with target 7, no pair adds to 7. After checking every distinct pair, the function prints **-1 -1**.

Complexity

Time Complexity: $O(n^2)$ in the worst case because the program checks pairs using nested loops.

Space Complexity: O(1) extra space because no additional array or data structure is required.

Result

The program successfully finds two distinct indices whose values add up to the target using pointer-based array access and nested loops. If no valid pair exists, it correctly displays -1 -1.

Execution Example

```
Enter the size of array: 4
Enter the elements: 2 7 11 15
Enter target: 9
0 1
```